

User Manual

Safe Semantic Planner — PCCST503 Assignment 1

1. Requirements

Any C++17 compiler and a standard library. No third-party dependencies, no build system, no configuration.

Tested with g++ 13.3.0 on Linux x86-64. clang++ 15 and MSVC 19.3 also accept the source.

2. Building

```
g++ -O2 -std=c++17 -Wall -Wextra -Wpedantic -o safe_planner safe_planner.cpp
```

The build is warning-free. Add `-g -fsanitize=address,undefined` while developing; the test suite runs clean under both sanitizers.

A C11 version of the same design is also included and builds with:

```
gcc -O2 -std=c11 -Wall -Wextra -o safe_planner_c safe_planner.c -lm
```

3. Running

Command	What it does
<code>./safe_planner</code>	six test cases and the lattice benchmark (default)
<code>./safe_planner --tests</code>	the six assignment test cases only
<code>./safe_planner --bench</code>	the 150×150 lattice benchmark only
<code>./safe_planner --sweep</code>	cost/clearance trade-off as λ varies
<code>./safe_planner --scaling</code>	scaling study across five problem sizes
<code>./safe_planner --demo</code>	live ASCII demonstration of incremental replanning
<code>./safe_planner --demo --slow</code>	the same, paused ~0.9 s between frames for an audience
<code>./safe_planner --solve FILE</code>	plan on a problem described in a file
<code>./safe_planner --all</code>	every mode except the demo
<code>./safe_planner --help</code>	usage summary

Exit status is 0 on success, 1 if `--solve` finds no path, 2 on a usage or input error.

Runtime for the default run is about 1 second; `--scaling` takes about 3 seconds.

4. Reading the output

```
result          : SUCCESS
path            : S -> C -> D -> G
total cost      : 3.600
effective cost   : 3.645
bad states visited : 0
min dist to bad  : 2.000
cum. reliability : 2.910 (product 0.9127)
Score(P)        : 119.310
states expanded  : 4
heap operations  : 7
planning time    : 0.0003 ms
planner memory   : 0.5 KB
optimality check : PASS (Dijkstra = 3.645, D* Lite = 3.645)
```

Field	Meaning
total cost	sum of nominal transition costs — what the plan costs in the world
effective cost	cost under the safety-weighted metric — what the search minimised
min dist to bad	smallest Euclidean distance from any visited state to any bad state; n/a when <i>B</i> is empty
cum. reliability	sum of per-transition reliabilities; the product is the probability the whole plan executes
Score(P)	$\alpha G - \beta C + \gamma D + \delta R$ from the brief, computed on the returned plan
states expanded	vertex expansions during this search — the work actually done
planning time	search only; graph setup and clearance construction excluded
optimality check	independent verification against DijkstraPlanner

effective cost is always at least **total cost**; the gap is the safety, reliability and per-transition-safety penalties the search charged.

5. Problem file format

Plain text, one directive per line, **#** starts a comment. Order does not matter except that **state** lines must come before the **edge**, **bad**, **start** and **goal** lines that reference them.

```
dim      <d>
state    <id> <label> <x1> ... <xd>
bad      <id> [<id> ...]
edge     <from> <to> <cost> [safety] [reliability] [available]
start    <id>
goal     <id>
weights  <lambda> <dSafe> <mu> <nu> <alpha> <beta> <gamma> <delta>
```

- `dim` defaults to 2 and must precede the `state` lines.
- State ids must be consecutive from 0. The loader rejects gaps.
- `safety` and `reliability` default to 1.0, `available` to 1.
- `weights` is optional; omitting it uses the defaults of §6.
- Multiple `bad` lines accumulate.

A complete worked example ships as `warehouse.graph`:

```
dim 2
state 0 dock 0.0 0.0
state 1 aisleA 3.0 0.5
...
bad 8 9
edge 0 1 2.0 0.9 0.99
...
start 0
goal 7
weights 4.0 2.5 0.5 0.5 100.0 1.0 10.0 1.0
```

Run it with:

```
./safe_planner --solve warehouse.graph
```

Errors report the offending line number, for example `error: unknown keyword 'frobnicate' on line 3`.

6. Tuning the objective

Parameter	Default	Effect
<code>lambda</code>	4.0	how much cost to pay for clearance. 0 minimises cost alone
<code>dSafe</code>	2.0	clearance beyond which no penalty is charged
<code>mu</code>	0.5	penalty weight on unreliability, $\mu(1 - r)$
<code>nu</code>	0.5	penalty weight on per-transition unsafety, $\nu(1 - \sigma)$
<code>alpha,beta,gamma,delta</code>	100, 1, 10, 1	reporting coefficients for <code>Score(P)</code> ; they do not influence the search

Practical guidance:

- λ **has the units of cost**. Since the penalty is normalised by d_{safe} , a λ of 4.0 means “I will pay up to 4 cost units per state to go from touching a hazard to being clear of it”. Set it by asking what that trade is worth in your domain.
- d_{safe} **is a threshold, not a target**. Beyond it the penalty is exactly zero, so the planner will not spend cost buying clearance it does not need.

- **The relationship between λ and the chosen route is a step function, not a slope.** Small changes do nothing until a crossover point, then flip the whole route. Use `--sweep` to find where the crossovers are for your graph before settling on a value.
- **Some compromise routes are unreachable by any λ .** See §4 of the experimental results: a linear objective can only return vertices of the lower convex hull of the (cost, penalty) trade-off. If a moderate route is what you want, a hard clearance constraint is the right tool, not parameter tuning.

7. Using the planner as a library

Include `safe_planner.cpp` or split out the `planning` namespace. Minimal use:

```
using namespace planning;

PlanningProblem prob;
prob.states = { /* State{id, embedding} ... */ };
prob.transitions = { /* Transition{...} ... */ };
prob.badStates = { 7 };
prob.initialState = 0;
prob.goalState = 12;

Weights w;                                // or set lambda, dSafe, mu, nu
DStarLitePlanner planner(prob, w);

PlanningResult r = planner.plan(prob);
if (r.success) {
    for (std::uint64_t s : r.statePath) { /* execute */ }
}
```

7.1 Replanning after a change

Call the matching mutator, then `plan()` again. Never rebuild the planner — that discards the search tree and forfeits the entire benefit.

```
planner.setTransitionAvailable(edgeId, false);    // blocked
planner.setTransitionCost(edgeId, 7.5);          // cost changed
planner.addTransition(from, to, cost, safety, reliability);
planner.setStart(currentState);                  // agent moved
planner.setGoal(newGoal);                        // goal moved
planner.setBadStates({3, 9, 14});                // hazards changed
r = planner.plan(prob);                          // repairs only what changed
```

A typical execute-and-replan loop:

```
PlanningResult r = planner.plan(prob);
while (r.success && r.statePath.size() > 1) {
    std::uint64_t next = r.statePath[1];
    move(next);
    planner.setStart(next);
    for (std::uint64_t e : sensedBlockedEdges())
        planner.setTransitionAvailable(e, false);
}
```

```

    r = planner.plan(prob);
}

```

Moving the agent without any environment change costs zero expansions, so calling `plan()` every step is cheap.

7.2 Rules and constraints

- **One planner, one problem.** `plan()` throws `std::invalid_argument` if given a different `PlanningProblem` instance. Incremental state is meaningless across problems.
- **Transitions can be added at runtime; states cannot.** The per-state arrays are sized at the first `plan()` call. Add every state before planning.
- **Changing Weights requires a new planner.** Weights alter every edge weight in the graph, so no incremental repair applies.
- **Mutate through the planner, not the problem.** Editing `prob.transitions` directly leaves the search tree stale. The mutators exist to mark the right vertices dirty.
- **Bad-state changes are the most expensive update**, because the clearance field is rebuilt in $O(nkd)$. Batch them into a single `setBadStates` call rather than calling it per hazard.

8. Extending

Goal	Where to change
Different cost model	<code>SafetyModel::weight</code> — the single place edge weights are computed; keep $w \geq c$ or the heuristic stops being admissible
Non-Euclidean embedding	<code>euclid()</code> and <code>SafetyModel::recomputeClearance</code> ; η in <code>minCostPerLength</code> must use the same metric
Sparse or string ids	add <code>unordered_map<Id, uint32_t></code> in the loader; the planner already works purely in indices
Multi-goal	add a virtual goal state with zero-cost transitions from each real goal; no change to the search
Faster clearance	replace the double loop in <code>recomputeClearance</code> with a k-d tree query
Hard clearance constraint	in <code>SafetyModel::weight</code> , return ∞ when $\text{clear}(v) < d_{min}$

9. Troubleshooting

Symptom	Cause	Fix
<code>result : FAILURE</code>	goal unreachable, or every route blocked by bad states or unavailable transitions	check <code>bad</code> lines and <code>available</code> flags; try $\lambda = 0$ to see whether safety weighting is the cause
<code>optimality check : FAIL</code>	heuristic overestimating	ensure any custom weight function satisfies $w \geq c$

Symptom	Cause	Fix
Path avoids bad states but hugs them	λ too small or d_{safe} too small	raise both; use --sweep to locate the crossover
Replanning slower than expected	the change invalidated much of the tree	expected — see §5.2 of the experimental results; consider a fresh planner for wholesale changes
<code>std::invalid_argument</code> from <code>plan()</code>	planner bound to a different problem instance	construct the planner with the problem you intend to plan on
<code>error: state ids must be consecutive from 0</code>	gap or out-of-order <code>state</code> line	renumber the states in the file